

SQL GROUP BY Concepts & Functions

Group by Dates?

Suppose we have an Orders table.

order_id	customer	order_date	amount
1	Arpit	2026-01-02	1200
2	Aadz	2026-01-02	900
3	Arpit	2026-01-03	1500
4	Rahul	2026-02-10	700
5	Aadz	2026-02-18	1300
6	Arpit	2026-03-05	2000

Question

How much revenue did we earn every month?

Instead of grouping by customer, we group by time.

14.14.2 Daily Reports

Daily revenue.

```
SELECT DATE(order_date) AS day,  
       SUM(amount) AS revenue  
FROM Orders  
GROUP BY DATE(order_date)  
ORDER BY day;
```

GROUP BY EXTRACT()

PostgreSQL standard.

```
SELECT EXTRACT(YEAR FROM order_date),  
       SUM(amount)  
FROM Orders  
GROUP BY EXTRACT(YEAR FROM order_date);
```

Practical Dashboard Examples

Daily active users.

```
SELECT DATE(login_time), COUNT(*)  
FROM LoginHistory  
GROUP BY DATE(login_time);
```

Monthly registrations.

```
SELECT YEAR(join_date), MONTH(join_date), COUNT(*)  
FROM Users  
GROUP BY YEAR(join_date), MONTH(join_date);
```

Yearly admissions.

```
SELECT YEAR(admission_date), COUNT(*)  
FROM Students  
GROUP BY YEAR(admission_date);
```

GROUP BY String Functions

★★★★★ Frequently used in reporting and data cleaning
SQL allows grouping using expressions, not just columns.

String functions are particularly useful for:

- categorization
- domain analysis
- initials
- prefixes
- text normalization

14.15.1 Why Group by Strings?

Customers

Customer
Arpit
Aadz
Rahul
Riya
Priya

Question

Count customers by first letter.

Need

A → Arpit, Aadz

R → Rahul, Riya

etc.

14.15.2 GROUP BY LEFT()

MySQL

```
SELECT LEFT(customer_name, 1) AS initial,
       COUNT(*) AS customers
FROM Customers
GROUP BY LEFT(customer_name, 1)
ORDER BY initial;
```

Output

Initial	Customers
A	2
P	1
R	2

GROUP BY RIGHT()

Suppose product codes.

Code
ABC101
XYZ101
AAA205

Group by last three digits.

```
SELECT RIGHT(product_code, 3), COUNT(*)
FROM Products
GROUP BY RIGHT(product_code, 3);
```

GROUP BY UPPER()

Suppose

City
Delhi
DELHI
delhi

Without normalization

Three groups.

With

```
SELECT UPPER(city), COUNT(*)
FROM Customers
GROUP BY UPPER(city);
```

Output

City	Count
DELHI	3

Very useful for dirty datasets.

Group Emails by Domain

Users

Email
arpit@gmail.com
aadz@gmail.com
rahul@yahoo.com

Extract domain.

```
SELECT SUBSTRING_INDEX(email, '@', -1) AS domain,  
       COUNT(*) AS users  
FROM Users  
GROUP BY domain;
```

Output

Domain	Users
gmail.com	2
yahoo.com	1

Salary Buckets

```
SELECT  
  CASE  
    WHEN salary < 50000 THEN 'Low'  
    WHEN salary < 100000 THEN 'Medium'  
    WHEN salary < 200000 THEN 'High'  
    ELSE 'Executive'  
  END AS salary_range,  
  COUNT(*) AS employees  
FROM Employee  
GROUP BY salary_range  
ORDER BY salary_range;
```

Sales Analytics Example

```
SELECT
  CASE
    WHEN amount < 1000 THEN 'Small'
    WHEN amount < 5000 THEN 'Medium'
    ELSE 'Large'
  END AS order_size,
  COUNT(*) AS orders,
  SUM(amount) AS revenue
FROM Orders
GROUP BY order_size;
```

What is bucketization?

Grouping continuous numeric values into meaningful ranges.

Why use CASE with GROUP BY?

To create custom categories that don't exist as stored columns.